Log Analytics System for Web Server Logs: Hadoop HDFS, Apache Spark, Spark MLlib and Apache Superset for Traffic, Bot and Attack Analysis

Kushal Mohan
Department of Networking and Communications, School of Computing
SRM Institute of Science and Technology
Kattankulathur, Chennai, India
RA2411028010082

Kriti Singh
Department of Networking and Communications, School of Computing
SRM Institute of Science and Technology
Kattankulathur, Chennai, India
RA2411028010081

Gargi Singh
Department of Networking and Communications, School of Computing
SRM Institute of Science and Technology
Kattankulathur, Chennai, India
RA2411028010102

Dr. Senthil Kumar K*
Department of Networking and Communications, School of Computing
SRM Institute of Science and Technology
Kattankulathur, Chennai, India
Corresponding author (project guide)

***Abstract*—Web server access logs record every request a site receives, but at millions of lines per day they are too large and too unstructured for ad-hoc scripts, which also tend to drop malformed lines silently. This paper presents the Log Analytics System, a reproducible pipeline that stores logs in the Hadoop Distributed File System (HDFS), cleans them with Apache Spark, analyses them with Spark SQL, mines them with Spark MLlib and presents every result through big data visualisation tools: Apache Superset dashboards served by the Spark Thrift Server, embedded in an application called LogLens. Everything runs on a ten-container Docker cluster with two DataNodes and a replication factor of two. On the NASA-HTTP trace (3,461,613 requests), the Spark ETL keeps 3,461,612 typed records in date-partitioned Parquet, 8.6 times smaller than the raw text, and preserves the single malformed line. Eleven Spark SQL reports characterise the traffic, and a contextual Z-score detector flags twelve unusual hours, eleven of them on the morning of the STS-70 Space Shuttle launch. An unsupervised Gaussian Mixture Model over nine per-host behavioural features isolates 1,829 automated hosts (6.36 % of requests) with an 11.6-fold lift on known crawlers. A Factorization Machine trained on 1.3 million labelled HTTP requests reaches an F1 of 0.967 on 261,076 unseen requests, within 0.002 of the best of seven MLlib baselines, and transfers best to an unseen dataset (F1 0.968 against 0.897 and 0.846). We also report its limits: inside one dataset a linear SVM is significantly better on four of five random splits, and on the unlabelled 1995 logs none of the 120 flagged URLs is a real attack.**

***Index Terms*—log analytics, Hadoop HDFS, Apache Spark, Spark SQL, Spark MLlib, Apache Superset, anomaly detection, Gaussian mixture model, factorization machines, web-attack detection, dataset bias**

I. Introduction

Every request to a web server becomes one line in its access log. Those lines hold the answers to practical questions: which pages are popular and which are broken, when traffic peaks, which clients are programs rather than people, and which requests try to attack the site. A busy server, however, writes millions of lines a day as plain text with no queryable structure. Early studies of web workloads ran on single machines [1], [2], and production log platforms that do scale assume large clusters and dedicated teams [3].

This paper describes a complete, reproducible system that applies the distributed approach end to end, from raw text to an interactive application, and uses it to answer three analytical questions on a real trace: when did traffic behave unusually, which clients are automated, and how well does a machine-learning detector trained on public attack data recognise attacks? A course constraint shaped the presentation layer: every chart must come from a big data visualisation tool rather than hand-written charting code.

The contributions are: (1) a pipeline on HDFS, Spark, Spark SQL and Spark MLlib that reports its own data quality instead of hiding it, started by one command on a ten-container cluster; (2) a contextual Z-score detector in Spark SQL that recovers a real event, the STS-70 launch, from the traffic alone; (3) an unsupervised Gaussian Mixture Model that separates automated clients using only Common Log Format fields; (4) a Factorization Machine for web-attack detection, compared with seven MLlib classifiers on a mixed split, on five random splits with McNemar's test, and across datasets, after measuring and removing a shortcut in the public data; and (5) a presentation layer built only from big data visualisation tools: Apache Superset dashboards querying HDFS through the Spark Thrift Server, embedded in the LogLens application, which also runs the saved Spark ML model on requests typed by a user.

II. Related Work

Web log mining. Arlitt and Williamson characterised six web-server workloads, including the NASA logs used here, with single-machine tools [1]. Cooley et al. showed that raw logs need careful cleaning before analysis [2], and Srivastava et al. framed web usage mining as preprocessing, pattern discovery and pattern analysis [4]. Oliner et al. surveyed log analysis and named volume and messy formats as its main obstacles [3].

Big data platforms. MapReduce [5] and the Google File System [6] showed how to process and store data across many cheap machines; HDFS is the open-source descendant of that design [7]. Spark keeps working data in memory [8], Spark SQL adds relational queries with an optimiser [9], and column

storage in the style of Dremel [10] lets a query read only the columns it needs. The Spark Thrift Server exposes Spark SQL over JDBC and ODBC [11], which lets business-intelligence tools such as Apache Superset [12] query data that stays in HDFS. Containers make such a cluster reproducible on one machine [13].

Anomaly, bot and attack detection. Statistical scores work well when normal behaviour can be modelled [14]. Xu et al. mined console logs with PCA [15] and DeepLog learned normal log sequences with an LSTM [16]; both target system logs rather than web traffic. Gaussian mixtures model populations made of a few sub-groups of different shape and size [17]. For malicious URLs, character-level models learn from raw strings [18]. Most attack-detection studies report one mixed train/test split; shortcut learning, where a model exploits an incidental cue [19], is rarely measured. Factorization Machines [20] model pairwise feature interactions and come from recommender systems; we are not aware of their use for web-attack detection on Spark.

III. System Architecture

The system runs from two Docker images: one with Java 11, Hadoop 3.3.6, Spark 3.5.3 and Python for every cluster role, and the official Apache Superset 4.1.1 image with a Hive/Spark driver. Docker Compose starts ten containers (Table I); an entrypoint script gives each container its role, and every component reaches HDFS at hdfs://namenode:8020. Fig. 1 shows the data flow.

TABLE I
CONTAINERS OF THE CLUSTER

Container	Role	Resources / ports
namenode	HDFS metadata: files, blocks, locations	UI 9870, RPC 8020
datanode1, datanode2	HDFS block storage, own volume each	replication 2, 128 MB blocks
spark-master	Spark standalone cluster manager	7077, UI 8080
spark-worker	Runs all Spark executors	3 cores, 3 GB
client	hdfs commands and spark-submit	mounts code and data
thrift	Spark Thrift Server (Spark SQL over JDBC)	10000; 1 core, 1 GB
superset	Apache Superset dashboards, SQL Lab	8088
app	LogLens application	3080

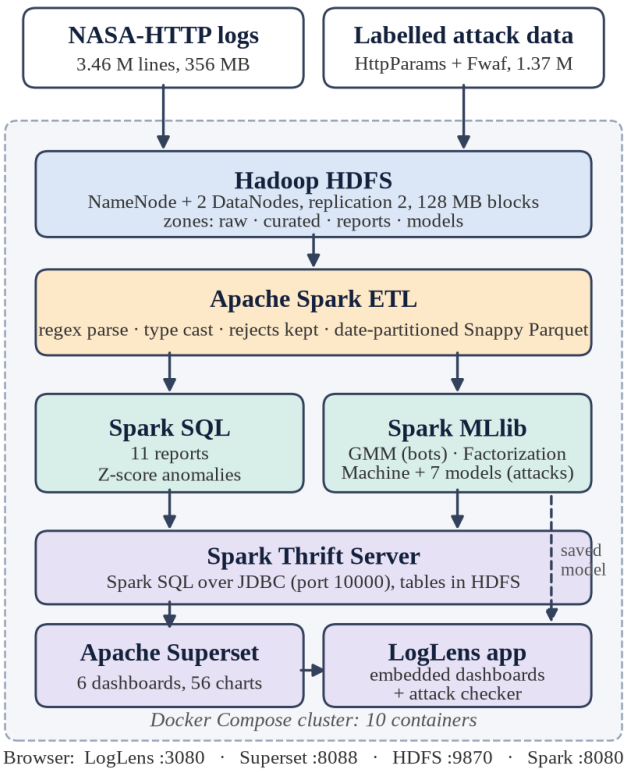


Fig. 1. Architecture. Raw logs and labelled attack data enter HDFS; Spark turns them into Parquet tables; Spark SQL and Spark MLlib analyse them; the Spark Thrift Server serves the tables to Apache Superset, whose dashboards are embedded in the LogLens application together with the saved attack-detection model.

Data moves through four HDFS zones, each written by one stage and never edited by a later one: *raw* (the files as downloaded, the audit trail), *curated* (typed Snappy Parquet, one folder per date), *reports* (Spark SQL, anomaly, bot and ML result tables) and *models* (the saved Spark ML pipeline). Because the raw zone is immutable, every result can be rebuilt from it. Replication is checked with hdfs fsck, which reports two locations for every block.

IV. Data and Preprocessing

A. NASA-HTTP Logs

The NASA-HTTP trace [21] holds two months (July–August 1995) of requests to NASA Kennedy Space Center's web server in Common Log Format: host, timestamp, request line, status and bytes. Logs are missing for 29–31 July and the server was down from 1 August 14:52 to 3 August 04:36; these gaps are kept, not filled.

The Spark ETL reads the raw zone as one text column, cuts each line into seven fields with five regular expressions, casts them (timestamp, integer status, long bytes, with a dash read as zero bytes) and derives date, hour, weekday and file extension. A line without a valid timestamp, status or host goes to a rejects folder instead of being dropped, and the parse rate is printed on every run (Table II). Timestamps use the server's own time zone (UTC−4).

TABLE II
ETL RESULT: RAW ZONE AND CURATED ZONE

Measure	Raw zone	Curated zone
Rows	3,461,613 lines	3,461,612 + 1 reject
Columns	1 (text)	11 typed
Format	plain text, 2 files	Snappy Parquet, 58 date folders
Size	356 MB	41.5 MB (8.6× smaller)

The single reject is a truncated line that reads only *alyssa.p*; keeping it visible is what makes the parse rate of 100.0 % checkable.

B. Labelled Attack Data

The NASA logs carry no attack labels, so the classifiers learn from two public datasets stored in HDFS: HttpParams [22], 31,067 parameter values labelled normal, SQL injection, cross-site scripting, command injection or path traversal; and Fwaf [23], 1,294,531 normal and 48,126 malicious URLs. All attack types are merged into one class, giving a binary task with two classes (0 = normal, 1 = attack).

Preprocessing URL-decodes, lower-cases and trims each request, then removes duplicates and requests that appear with both labels. Inspecting the data revealed a shortcut: 42.2 % of attack requests but only 2.5 % of normal ones contain a question mark, because Fwaf's normal examples are mostly bare paths, so a model can score well by learning that a query string means an attack. We therefore keep only the part after the first question mark, so the model must judge what is sent; afterwards the shares are 2.7 % and 0.8 %. Table III lists every step.

TABLE III
PREPROCESSING OF THE LABELLED DATA

Step	Rows
Loaded and labelled	1,373,724
After cleaning (67 empty rows dropped)	1,373,657
Duplicates removed	66,987
Conflicting labels removed	501
Final dataset (97.06 % normal, 2.94 % attack)	1,306,017
Training set: all attacks + 4 normal per attack	153,847
Test set: real class mix, never used in training	261,076

V. Traffic Analytics

A. Spark SQL Reports

The curated table is registered as a view and every question is one SQL query. Eleven reports cover traffic (by hour, day and weekday), content (top URLs, content types), errors (status mix, daily 404 rate, broken links) and response weight (size percentiles with percentile_approx [24], size buckets, heaviest hosts). Each report is written back to HDFS as Parquet. Table IV lists the main findings.

TABLE IV
MAIN FINDINGS OF THE SPARK SQL REPORTS

Finding	Value
Requests · distinct hosts · bytes served	3,461,612 · 137,978 · 61.02 GB
Successful responses (200)	89.6 %
Not found (404)	20,899 requests (0.6 %)
Most-visited page	/ksc.html, 79,993 hits
Worst broken link	/pub/winvn/readme.txt, 2,004
Busiest hour / weekday	15:00 (230,665) / Thursday (667,737)
Median / 99th-percentile response	4,179 B / 283,389 B

B. Contextual Z-Score Anomaly Detection

Requests are counted for every (date, hour) slot, and each slot is compared only with the same hour of day on other days, so a busy afternoon is not flagged merely for being an afternoon:

$$z(d,h) = (n(d,h) - \mu(h)) / \sigma(h) \quad (1)$$

where n(d,h) is the request count on date d at hour h, and μ(h) and σ(h) are the mean and population standard deviation of that hour across all days. A slot with |z| > 3 is flagged. The detector is four lines of Spark SQL and needs no labels.

It flags 12 hours, 11 of them on 13 July 1995 (Fig. 2). The strongest is 09:00–10:00 that day: 14,926 requests against a normal 3,080 (σ = 2,006), z = 5.90, the hour of the STS-70 launch at 9:41 a.m. On daily totals only 13 July is flagged (z = 3.62). The twelfth hour is 31 August 04:00 (z = 3.05). Outage hours have no rows and therefore appear as gaps, not as drops.

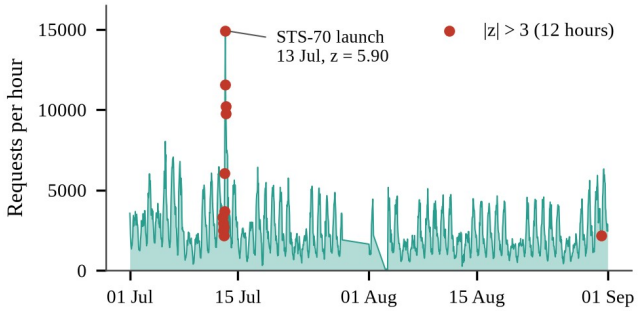


Fig. 2. Requests per hour over the two months. Red points are the 12 hours with |z| > 3; 11 of them fall on 13 July 1995, the STS-70 launch.

VI. Bot Detection with a Gaussian Mixture Model

Common Log Format has no user-agent field, so automated clients must be recognised from behaviour alone. For each of the 38,104 hosts with at least 20 requests, Spark splits the requests into sessions (30-minute inactivity gap) and computes nine features: share of HEAD requests, median gap between requests, share of images, number of sessions, requests per session, number of top-level directories, share of unique URLs, error share and the coefficient of variation of the gaps. Heavy-tailed counts are log-transformed and all features standardised.

A Gaussian Mixture Model [17] with three components is fitted in Spark MLlib (silhouette 0.49 [25]). The component with the most HEAD requests, the slowest pacing and the fewest images is labelled automated from its own parameters, without any labels (Table V, Fig. 3). We chose a GMM over K-Means because the automated group is small and elongated: with k = 2, K-Means absorbed it into a larger cluster, and with k = 8 it split it into fragments.

TABLE V
THE THREE GMM COMPONENTS

Component	Hosts	Requests	HEAD %	Gap (s)	Images %
C0 human	32,503	52.3 %	0.04	5	63.2
C1 human	3,772	41.4 %	0.10	2	70.2
C2 automated	1,829	6.4 %	2.50	18	33.5

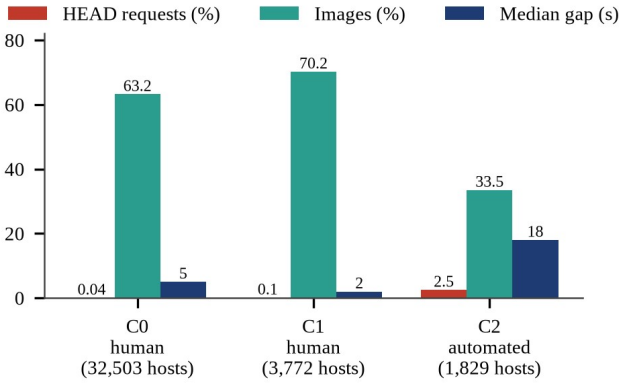


Fig. 3. Behaviour of the three GMM components. The automated component uses HEAD requests, waits longer between requests and loads fewer images.

As a held-out check, 27 hosts requested /robots.txt, a strong sign of a crawler. The automated component contains 15 of them (recall 55.6 %), 11.6 times the rate expected by chance. Because most crawlers never request robots.txt, this recall is a lower bound.

VII. Attack Detection Method

A. Features

Each request is split into characters, from which Spark builds character 1-, 2- and 3-grams. Each n-gram size is hashed into 4,096 buckets and four counts are appended: length, special characters, digits and attack keywords (for example union, <script, ../). This gives 12,292 sparse features, scaled to [0, 1] with MaxAbsScaler so the vectors stay sparse. The feature pipeline is fitted on the training set only.

B. Factorization Machine

A Factorization Machine [20] extends logistic regression with a weight for every pair of features, factorised through k-dimensional vectors so that pairs never seen together in training still receive a weight:

$$\hat{y}(x) = \sigma\left(w_0 + \Sigma_i w_i x_i + \Sigma_i \Sigma_{j(j>i)} \langle v_i , v_j \rangle x_i x_j \right) \qquad (2)$$

where σ is the logistic function and $v_i \in \mathbb{R}^k$ is the factor vector of feature i. The pairwise term costs O(kn), linear in the number of non-zero features. For attack detection it lets the model weigh combinations, such as a quote together with *or* and a comment marker, rather than each n-gram alone. We use Spark MLlib's FMClassifier [26]. Its settings, and the regularisation of the linear baselines, were chosen on a validation split carved from the training set (122,910 fitting and 30,937 validation requests; the test set is never read): factor size k = 8 and step size 0.2 won over $k \in \{4, 16\}$ and step 0.05 (validation F1 0.988), with 100 iterations and L2 regularisation 10^{-4}.

C. Comparison Models and Protocol

All eight models are trained in Spark MLlib on the same features: the Factorization Machine, a linear SVM [27] and logistic regression (100 iterations, L2 10^{-4}), multinomial Naive Bayes, a decision tree (depth 10), a random forest [28] (30 trees, depth 10), gradient-boosted trees [29] (15 rounds, depth 5) and a multilayer perceptron (388–32–2; Spark's MLP needs dense input, so it uses a 388-feature version of the same n-grams). We report precision, recall and F1 for the attack class with the confusion matrix; since 97 % of test requests are normal, accuracy alone is misleading and PR-AUC is reported as the area metric [30].

The protocol has four parts: (i) one mixed 80/20 split (seed 42); (ii) the top three models on five further random splits, with mean ± standard deviation and McNemar's test [31], [32] on each split; (iii) a cross-dataset test, training on one dataset and testing on the other with five seeds; and (iv) a case study scoring every distinct NASA URL with the deployed model.

VIII. Results

A. Mixed Split

On the 261,076 held-out requests (253,411 normal, 7,665 attack), all results being from the Docker cluster, the Factorization Machine reaches F1 = 0.9669, within 0.002 of the linear SVM and ahead of the tree, probabilistic and neural models (Table VI, Fig. 4). It catches 98.5 % of attacks with 399 false alarms (Table VII).

TABLE VI
EIGHT SPARK MLLIB MODELS ON THE SAME TEST SET

Model	Prec.	Rec.	F1	FP	FN	Time (s)
Linear SVM	0.9494	0.9885	0.9686	404	88	4.7
Factorization Machine (ours)	**0.9498**	**0.9846**	**0.9669**	**399**	**118**	**67.4**
Logistic Regression	0.9456	0.9855	0.9651	435	111	3.6
Decision Tree	0.9292	0.9586	0.9437	560	317	27.4
Gradient Boosted Trees	0.8190	0.9632	0.8853	1,632	282	131.6
Naive Bayes	0.9146	0.8033	0.8553	575	1,508	0.5
Multilayer Perceptron	0.7787	0.9387	0.8512	2,045	470	43.6
Random Forest	0.9571	0.7109	0.8158	244	2,216	24.5

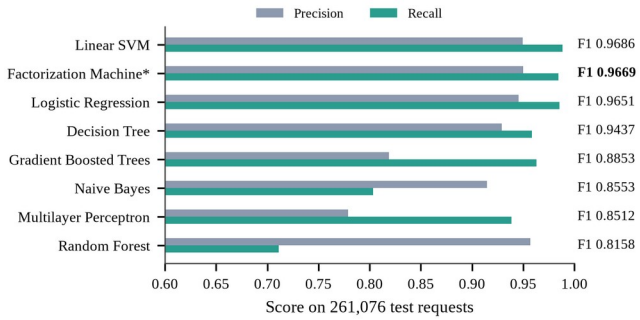


Fig. 4. Precision and recall of the eight models on the test set, sorted by F1. Accuracy (above 0.99 for every model) separates them poorly; F1 spreads them from 0.816 to 0.969.

TABLE VII
CONFUSION MATRIX OF THE FACTORIZATION MACHINE

	Predicted normal	Predicted attack
Actual normal	253,012 (TN, 99.84 %)	399 (FP, 0.16 %)
Actual attack	118 (FN, 1.54 %)	7,547 (TP, 98.46 %)

B. Stability over Five Random Splits

One split can be lucky, so the whole experiment (split, balancing, features, training) was repeated with seeds 1–5. Inside one dataset the linear SVM is the most accurate and the most stable model (F1 0.9697 ± 0.0007), followed by logistic regression (0.9663 ± 0.0009) and the Factorization Machine (0.9611 ± 0.0062), whose factors start from random values. McNemar's test finds the SVM's advantage over the Factorization Machine significant ($p < 0.05$) on four of five splits, and logistic regression's on three of five. All three keep a PR-AUC above 0.989. Seed 42 of Table VI was one of the Factorization Machine's better splits.

C. Across Datasets

A mixed split lets a model learn the style of each dataset. We therefore train on one dataset and test on the other with five seeds (Table VIII, Fig. 5). Trained on Fwaf and tested on HttpParams, the ranking reverses: the Factorization Machine keeps F1 0.968 on all five seeds, against 0.897 for logistic regression and 0.846 for the SVM, with 6–14 times fewer false alarms. Its factorised pairwise terms capture n-grams that occur together in attacks rather than dataset-specific single n-grams. Trained only on the small HttpParams set, every model fails on Fwaf's full URLs, and the Factorization Machine overfits most.

TABLE VIII
STABILITY AND CROSS-DATASET TRANSFER (F1)

Model	5 splits	Fwaf → HttpP.	HttpP. → Fwaf
Linear SVM	0.9697 ± 0.0007	0.846 ± 0.012	0.303 ± 0.010
Logistic Regression	0.9663 ± 0.0009	0.897 ± 0.011	0.411 ± 0.000
Factorization Machine (ours)	0.9611 ± 0.0062	0.968 ± 0.008	0.169 ± 0.045

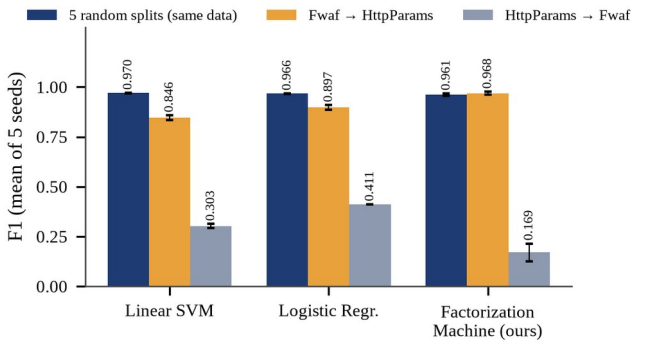


Fig. 5. F1 of the three strongest models inside one dataset (five random splits) and across datasets (five seeds each). Error bars are one standard deviation.

D. Case Study: the Unlabelled NASA Logs

The deployed model scored all 31,026 distinct NASA URLs after the same normalisation and flagged 120 (1,754 requests, 0.05 % of all traffic). Every flag was read by hand and sorted by rules written down in an audit script (Table IX). None contains a real attack signature such as union select, <script, ../.. or /etc/passwd. Most are site searches with words such as *select* or *transcript*, and pages under /msfc/description/, whose path shares character n-grams with <script. Without labels this is a qualitative check of the domain shift between modern attack data and 1995 traffic, not a precision measurement.

TABLE IX
HAND AUDIT OF THE 120 FLAGGED NASA URLS

What the URL really is	URLs	Requests
Site search (wais.pl)	44	55
Page under /msfc/description/	35	933
Malformed or mistyped request	21	38
Other ordinary page or file	11	708
CGI with parameters	9	20

IX. Visualisation and Application

The presentation layer uses only big data visualisation tools. The Spark Thrift Server runs on the cluster with one core and 1 GB, and a SQL script registers every HDFS folder as an external Spark table in a database named weblogs: the curated access_logs table (partitions discovered with MSCK REPAIR TABLE), the eleven reports, the anomaly and bot tables, and nine ML result tables published from the training jobs. Nothing is copied out of HDFS.

Apache Superset [12] connects to the Thrift Server and holds six dashboards with 56 charts: home, traffic, errors and performance, anomalies, bot detection and attack detection, the last with a native filter that switches the confusion matrix between models. Every chart is a Spark SQL query that runs on the cluster when the dashboard opens; results are cached for an hour because the tables change only when the pipeline runs. A build script creates the database connection, 32 datasets, the charts and the dashboards through Superset's REST API, so the whole presentation layer is reproducible from code. SQL Lab lets a reviewer run any Spark SQL query live (Fig. 6).

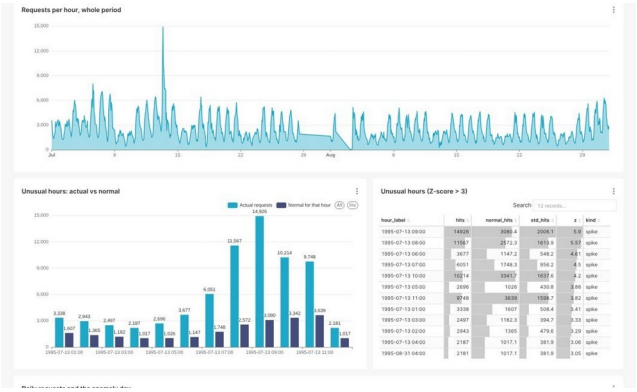


Fig. 6. The anomaly dashboard in Apache Superset: requests per hour over both months, the twelve unusual hours against their normal level, and the anomaly table, each a live Spark SQL query on HDFS.

LogLens, the application (Fig. 7), provides navigation, a home page and one page per analysis. It draws no charts itself: each page embeds a Superset dashboard in Superset's embedded mode, with a short-lived guest token, issued by the application's server, that can view only that one dashboard. The headline

numbers on the home page are Spark queries on HDFS. A *Check a request* page runs the saved Spark ML pipeline (n-grams, hashing, scaling and the Factorization Machine, stored in HDFS) on any URL a user types, in about 0.7 s, and explains the decision with the normalised text, the counts and the attack signals found. Reloaded from HDFS, the saved model reproduces F1 = 0.9669 on the test set.

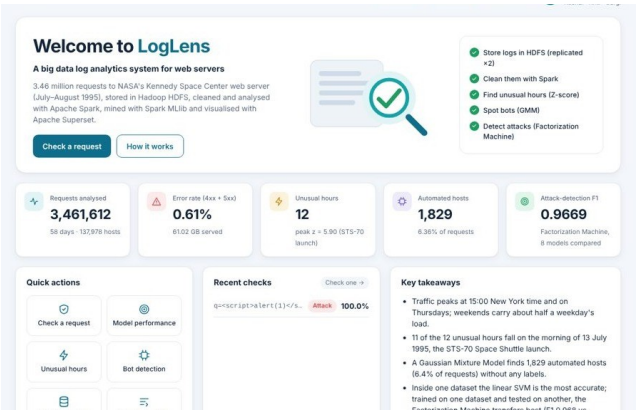


Fig. 7. LogLens home page: headline numbers computed by Spark on HDFS, quick actions and recent checks; the charts below them are an embedded Superset dashboard.

X. Reproducibility and Scalability

Every stage is one make target, and a single script starts the cluster and runs only the stages whose results are missing. A test suite of 35 tests runs every Spark job unchanged on a small sample with known answers: the exact parse of hand-written log lines, the one malformed line kept as a reject, report totals that add up, an injected traffic burst the Z-score must find, crawlers the GMM must separate, no request in both training and test sets, valid confusion matrices, a real Spark Thrift Server that must serve every dashboard table, and every page of the application. The same jobs run in a Google Colab notebook.

To measure scaling, the ETL and the eleven reports ran on 25 %, 50 %, 100 % and 200 % of the logs with one and two cores (Fig. 8). Doubling the data from 3.46 to 6.92 million lines raised the time from 123 s to 187 s on two cores (36,968 lines/s), because Spark's fixed start-up cost is spread over more lines, and a second core helped more as the data grew (speed-up 1.15 at 25 %, 1.47 at 200 %).

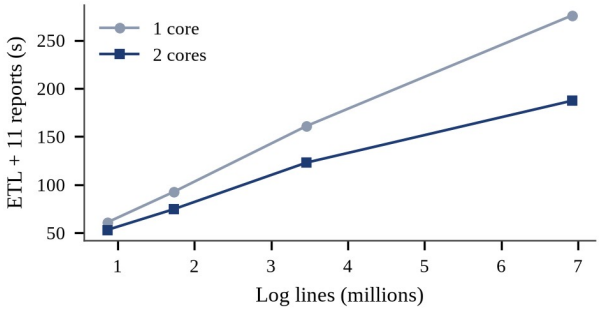


Fig. 8. Run time of the ETL and the eleven reports against data size, with one and two cores (Spark local mode, median of two runs).

XI. Discussion and Limitations

Mixed-split scores overstate what a detector knows. Before the bias fix the same models scored higher and an earlier model flagged 1,083 NASA URLs, mostly ordinary searches, because it had learned that a query string signals an attack. After the fix, the within-dataset ranking (SVM > logistic regression > FM, significant on most splits) and the cross-dataset ranking (FM ≫ logistic regression > SVM) disagree. We deploy the Factorization Machine because a detector always meets traffic it was not trained on, and report both results.

Limitations. The NASA logs have no labels, so detection on them is a qualitative case study. Both labelled datasets are small next to real traffic and come from neither the same server nor the same era. The cluster is distributed in software but runs on one laptop, so replication protects against a DataNode failing, not against the disk failing, and scaling was measured only up to 712 MB. The Factorization Machine's run-to-run variance (F1 standard deviation 0.006) is larger than that of the linear models, and the decision threshold of 0.5 was not tuned.

XII. Conclusion and Future Work

The Log Analytics System shows that a laptop-hosted HDFS and Spark cluster can store, clean, analyse, mine and present 3.46 million web log lines reproducibly, with every number traceable to a Spark job and every chart produced by a big data visualisation tool. Contextual Z-scores recovered a real event, a Gaussian Mixture Model separated automated clients without labels, and a Factorization Machine detected attacks with an F1 of 0.967 and transferred best across datasets, while a linear SVM remained more accurate within one dataset. Measuring and removing a dataset shortcut, and testing across datasets, changed which model looks best; we recommend both as standard practice.

Future work: add labelled traffic closer to real servers, such as CSIC 2010 [33]; hand-label a random sample of flagged URLs to estimate precision on real traffic; average several Factorization Machine runs to reduce their variance; tune the decision threshold on the validation split; and measure scaling on a multi-node cluster with 5 and 10 times the data. Code, data scripts and results are available at https://github.com/kushal040511/Web-server-log-analytics-system.

Acknowledgment

The authors thank Dr. Senthil Kumar K, Department of Networking and Communications, SRM Institute of Science and Technology, for guiding this project.

References

[1] M. F. Arlitt and C. L. Williamson, "Internet web servers: Workload characterization and performance implications," IEEE/ACM Trans. Networking, vol. 5, no. 5, pp. 631–645, 1997.

[2] R. Cooley, B. Mobasher, and J. Srivastava, "Data preparation for mining World Wide Web browsing patterns," Knowledge and Information Systems, vol. 1, no. 1, pp. 5–32, 1999.

[3] A. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," Communications of the ACM, vol. 55, no. 2, pp. 55–61, 2012.

[4] J. Srivastava, R. Cooley, M. Deshpande, and P.-N. Tan, "Web usage mining: Discovery and applications of usage patterns from web data," SIGKDD Explorations, vol. 1, no. 2, pp. 12–23, 2000.

[5] J. Dean and S. Ghemawat, "MapReduce: Simplified data processing on large clusters," in Proc. 6th USENIX Symp. Operating Systems Design and Implementation (OSDI), 2004, pp. 137–150.

[6] S. Ghemawat, H. Gobioff, and S.-T. Leung, "The Google file system," in Proc. 19th ACM Symp. Operating Systems Principles (SOSP), 2003, pp. 29–43.

[7] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, "The Hadoop distributed file system," in Proc. IEEE 26th Symp. Mass Storage Systems and Technologies (MSST), 2010, pp. 1–10.

[8] M. Zaharia et al., "Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing," in Proc. 9th USENIX Symp. Networked Systems Design and Implementation (NSDI), 2012, pp. 15–28.

[9] M. Armbrust et al., "Spark SQL: Relational data processing in Spark," in Proc. ACM SIGMOD Int. Conf. Management of Data, 2015, pp. 1383–1394.

[10] S. Melnik et al., "Dremel: Interactive analysis of web-scale datasets," Proc. VLDB Endowment, vol. 3, no. 1–2, pp. 330–339, 2010.

[11] Apache Software Foundation, "Spark SQL: Distributed SQL engine (Thrift JDBC/ODBC server)," Spark 3.5.3 documentation. [Online]. Available: https://spark.apache.org/docs/3.5.3/sql-distributed-sql-engine.html

[12] Apache Software Foundation, "Apache Superset documentation." [Online]. Available: https://superset.apache.org/docs/

[13] D. Merkel, "Docker: Lightweight Linux containers for consistent development and deployment," Linux Journal, vol. 2014, no. 239, Art. no. 2, 2014.

[14] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," ACM Computing Surveys, vol. 41, no. 3, pp. 1–58, 2009.

[15] W. Xu, L. Huang, A. Fox, D. Patterson, and M. I. Jordan, "Detecting large-scale system problems by mining console logs," in Proc. 22nd ACM SOSP, 2009, pp. 117–132.

[16] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly detection and diagnosis from system logs through deep learning," in Proc. ACM CCS, 2017, pp. 1285–1298.

[17] D. A. Reynolds, "Gaussian mixture models," in Encyclopedia of Biometrics, S. Z. Li and A. Jain, Eds. Boston, MA, USA: Springer, 2009, pp. 659–663.

[18] J. Saxe and K. Berlin, "eXpose: A character-level convolutional neural network with embeddings for detecting malicious URLs, file paths and registry keys," arXiv:1702.08568, 2017.

[19] R. Geirhos et al., "Shortcut learning in deep neural networks," Nature Machine Intelligence, vol. 2, no. 11, pp. 665–673, 2020.

[20] S. Rendle, "Factorization machines," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2010, pp. 995–1000.

[21] Internet Traffic Archive, "NASA-HTTP: Two months of HTTP logs from the KSC-NASA WWW server." [Online]. Available: https://ita.ee.lbl.gov/html/contrib/NASA-HTTP.html

[22] Morzeux, "HttpParamsDataset," GitHub repository. [Online]. Available: https://github.com/Morzeux/HttpParamsDataset

[23] F. Ahmad, "Fwaf: Machine learning driven web application firewall," GitHub repository, 2017. [Online]. Available: https://github.com/faizann24/Fwaf-Machine-Learning-driven-Web-Application-Firewall

[24] M. Greenwald and S. Khanna, "Space-efficient online computation of quantile summaries," in Proc. ACM SIGMOD, 2001, pp. 58–66.

[25] P. J. Rousseeuw, "Silhouettes: A graphical aid to the interpretation and validation of cluster analysis," J. Computational and Applied Mathematics, vol. 20, pp. 53–65, 1987.

[26] X. Meng et al., "MLlib: Machine learning in Apache Spark," J. Machine Learning Research, vol. 17, no. 34, pp. 1–7, 2016.

[27] C. Cortes and V. Vapnik, "Support-vector networks," Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[28] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[29] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," Annals of Statistics, vol. 29, no. 5, pp. 1189–1232, 2001.

[30] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, e0118432, 2015.

[31] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," Psychometrika, vol. 12, no. 2, pp. 153–157, 1947.

[32] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," Neural Computation, vol. 10, no. 7, pp. 1895–1923, 1998.

[33] C. Torrano Giménez, A. Pérez Villegas, and G. Álvarez Marañón, "HTTP data set CSIC 2010," Information Security Institute, CSIC, Madrid, Spain, 2010.